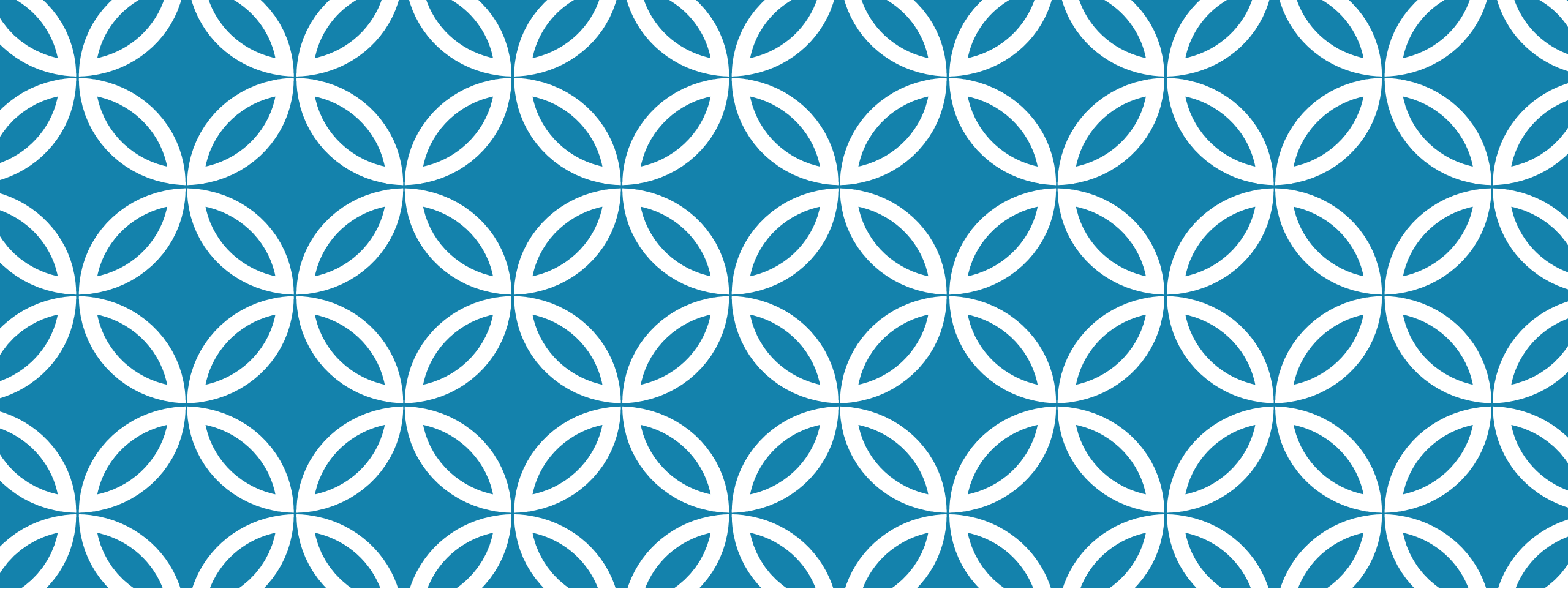




SEC3_DATA SCIENCE

Eng: Asmaa Farhat

SCIPY

Constants in SciPy

```
from scipy import constants  
print(constants.pi)
```

```
from scipy.constants import pi, e, mile
```

```
print(f"Pi: {pi}")  
print(f"Euler's number (e): {e}")
```

```
km = 5 * mile  
print(f"5 miles in kilometers: {km:.2f}")
```

SCIPY

```
# importing linalg function from scipy
from scipy import linalg
A= [[1, 2,], [5, 6,]]
# Compute the determinant of a matrix
linalg.det(A)
```

SCIPY

- Eigen values and Eigen vectors of this matrix

```
from scipy.linalg import eig

A = np.array([[6, 2], [2, 3]])

eigenvalues, eigenvectors = eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

SCIPY

Solving Linear Systems: Use `scipy.linalg` for advanced linear algebra operations.

```
from scipy.linalg import solve
```

```
A = np.array([[3, 2], [1, 2]])
```

```
b = np.array([5, 4])
```

```
x = solve(A, b)
```

```
print(f"Solution: {x}")
```



SCIPY

Computes the inverse of a square matrix.

```
from scipy.linalg import inv
import numpy as np

A = np.array([[3, 2], [1, -1]])

A_inv = inv(A)
print("Inverse of A:\n", A_inv)
```

SCIPY

Decomposes a matrix into **L (lower-triangular)**, **U (upper-triangular)**, and **P (permutation matrix)**.

```
from scipy.linalg import lu

A = np.array([[3, 2], [1, -1]])
P, L, U = lu(A)

print("P matrix:\n", P)
print("L matrix:\n", L)
print("U matrix:\n", U)
```

SCIPY

Numerical Integration: The `scipy.integrate` module provides tools to integrate functions.

```
from scipy.integrate import quad

# Define function to integrate
def f(x):
    return x**2

# Integrate f(x) from 0 to 1
result, error = quad(f, 0, 1)
print(f"Integration Result: {result}, Error: {error}")
```


SCIPY

1D and 2D Interpolation: `scipy.interpolate` can help with interpolation in one-dimensional and multi-dimensional data.

```
from scipy.interpolate import interp1d

# Sample data
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# Interpolate
f = interp1d(x, y, kind='quadratic')

# Use the interpolation function
print(f(2.5)) # This gives interpolated value at x = 2.5
```

SCIPY

```
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0, 2, 8, 18, 32, 50])

f = interp1d(x, y, kind='cubic') # Cubic spline interpolation

x_new = np.linspace(0, 5, 50)
y_new = f(x_new)

plt.plot(x, y, 'o', label='Data')
plt.plot(x_new, y_new, '-', label='Interpolated')
plt.legend()
plt.show()
```

SCIPY

Descriptive Statistics: Use scipy.stats for statistical analysis.

```
from scipy import stats

data = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]

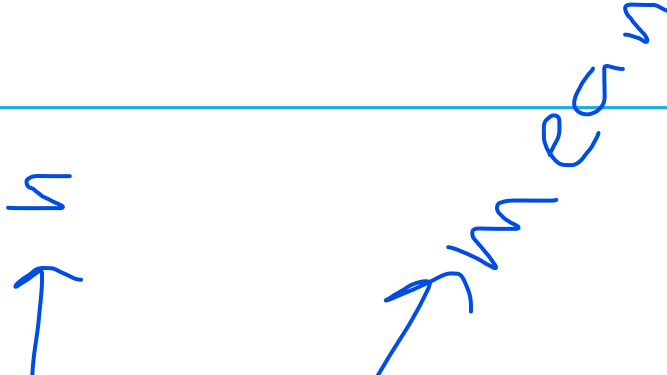
# Compute mean, median, mode
mean = np.mean(data)
median = np.median(data)
mode = stats.mode(data)

print(f"Mean: {mean}, Median: {median}, Mode: {mode.mode[0]}")
```

SCIPY

Generating Random Data

```
from scipy import stats
random_data = stats.norm.rvs(size=1000, loc=0, scale=1) # Normal distribution
```



SCIPY

Fourier Transform: `scipy.fft` is used to compute fast Fourier transforms.

```
from scipy.fft import fft
import numpy as np

# Sample data: a signal
t = np.linspace(0, 1, 1000)
signal = np.sin(2 * np.pi * 50 * t) + np.sin(2 * np.pi * 120 * t)

# Compute the Fourier Transform
signal_fft = fft(signal)
print(signal_fft)
```

SCIPY

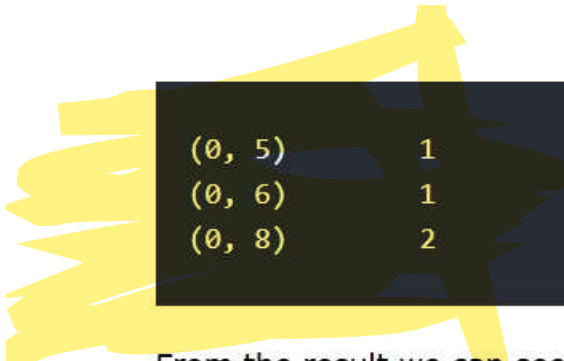
Sparse Matrices

Viewing stored data (not the zero items) with the **data** property:

```
import numpy as np
from scipy.sparse import csr_matrix

arr = np.array([0, 0, 0, 0, 0, 1, 1, 0, 2])

print(csr_matrix(arr))
```



```
(0, 5)    1
(0, 6)    1
(0, 8)    2
```

From the result we can see that there are 3 items with value.

The 1. item is in row **0** position **5** and has the value **1**.

The 2. item is in row **0** position **6** and has the value **1**.

The 3. item is in row **0** position **8** and has the value **2**.

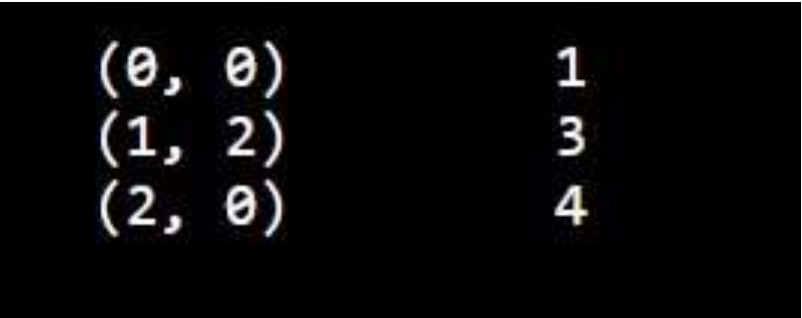
SCIPY

Sparse Matrices

SciPy supports sparse matrix operations, which can be useful for large datasets. Use `scipy.sparse` to create and work with sparse matrices.

```
from scipy.sparse import csr_matrix

# Create a sparse matrix
mat = csr_matrix([[1, 0, 0], [0, 0, 3], [4, 0, 0]])
print(mat)
```



(0, 0)	1
(1, 2)	3
(2, 0)	4

SCIPY

Hypothesis Testing with t-Test in SciPy:

This test checks if the mean of a sample is significantly different from a known population mean.

```
import numpy as np
from scipy import stats

scores = np.array([78, 82, 85, 90, 88, 75, 79, 81, 77, 80])

# Perform one-sample t-test
population_mean = 75
t_stat, p_value = stats.ttest_1samp(scores, population_mean)

print(f"T-Statistic: {t_stat:.4f}")
print(f"P-Value: {p_value:.4f}")
```


SCIPY

Hypothesis Testing with t-Test in SciPy: Comparing Test Scores of Two Different Classes

```
# Sample data: test scores from two different classes
class_A = np.array([85, 88, 75, 90, 95, 92, 78, 84, 89, 80])
class_B = np.array([78, 82, 79, 76, 85, 81, 83, 77, 74, 80])

# Perform independent t-test (equal variance assumed by default)
t_stat, p_value = stats.ttest_ind(class_A, class_B, equal_var=True)

print(f"T-Statistic: {t_stat:.4f}")
print(f"P-Value: {p_value:.4f}")
```

TASK

- Generate 5000 random samples from an exponential distribution with a mean of 3 and plot the histogram.
- Conduct a Chi-square goodness-of-fit test for a given dataset to check if it follows a normal distribution.
- Compute the confidence interval for a dataset using a 95% confidence level.

TASK

➤ Compute the Fourier Transform of the function $f(t) = \sin(2\pi t)$ using SciPy.

➤ Solve the system of equations:

$$\begin{cases} 3x + 2y = 18 \\ x - y = 2 \end{cases}$$

Compute the eigenvalues and eigenvectors of the matrix:

$$A = \begin{bmatrix} 4 & -2 \\ 1 & 1 \end{bmatrix}$$

Perform LU decomposition on the matrix:

$$B = \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix}$$